

第 14 组 软组织系统 软件体系结构设计

一、任务分工

任务	完成人
1、初步软件体系结构包图及文字描述	張振豐、陳俊皓
2、重用软件资源后得到的初步软件体系结构包图及文字描述	张二思
3、确定基础设施和服务后的软件体系结构包图及文字描述	董祉含
4、精化软件体系结构后的设计元素包图及文字描述	李峻安
5、部署图及文字描述	路良钧

二、初步软件体系结构

乐赛一站式服务平台的软件体系结构基于典型的 MVC（模型-视图-控制器）分层设计规范，由上至下构建了四个核心逻辑层，旨在通过严格的职责分离实现系统的高内聚与低耦合。



在最顶层的前端展示层中，系统利用响应式设计确保了跨终端的交互体验。该层主要通过三个职能明确的 UI 包来承载不同角色的业务需求：参赛选手 UI 包聚焦于赛事浏览、组队协同与个人荣誉展示；赛事主办方 UI 包则侧重于双轨赛事的发布与全流程管控；而平台管理员 UI 包提供了包含赛事审核、积分治理及项目投票在内的后端运营工作台。

核心业务逻辑由后端控制与服务层驱动。这一层级通过多个业务子包协作运行，其中用户与安全管理包负责基础的权限与身份控制，赛事核心业务包与报名与队伍协作包共同支撑起从赛事创建到组队报名的完整生命周期。特别地，积分与档案服务包承担了数据沉淀的重任，执行复杂的段位算法与荣誉更新；

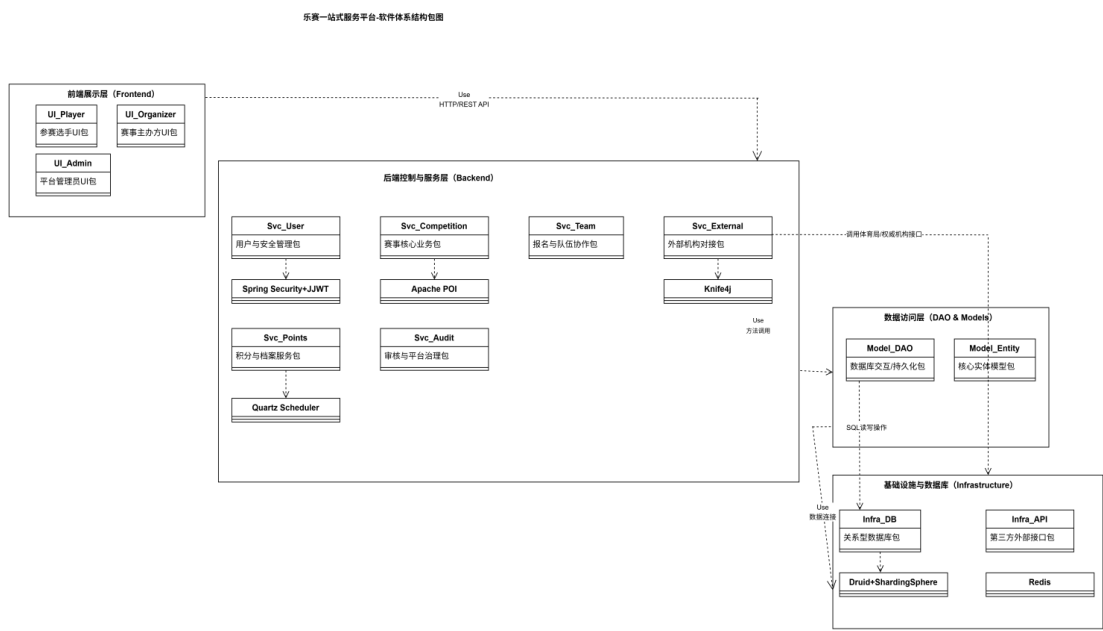
审核与平台治理包则保障了平台的合规运行。此外，外部机构对接包为后续与体育局等权威组织的联动预留了标准接口。

为了隔离业务逻辑与底层存储，数据访问层封装了系统的静态结构与持久化操作。该层中的核心实体模型包定义了用户、赛事、队伍等核心业务对象，而数据库交互包则通过标准的 CRUD 接口为上层服务提供透明的数据存取支持。

整个系统支撑在基础设施与数据层之上。关系型数据库包保障了用户档案与赛事数据的一致性与长期可靠存储，而第三方外部接口包则集成了外部认证或数据服务所需的环境，共同构成了乐赛平台稳定运行的技术底座。

三、重用软件资源后得到的初步软件体系结构

本系统在开发过程中重用了大量的开源软件资源，以提升开发效率与系统稳定性。



各开源包说明：

1. Spring Security + JWT

- 来源：Spring Security (<https://spring.io/projects/spring-security>)；JWT (<https://github.com/jwtkt/jjwt>)
- 获得方式：Maven 依赖
- 具体功能：实现用户认证与授权，支持 JWT 令牌生成与校验，区分选手、主办方、管理员角色，防止越权访问。

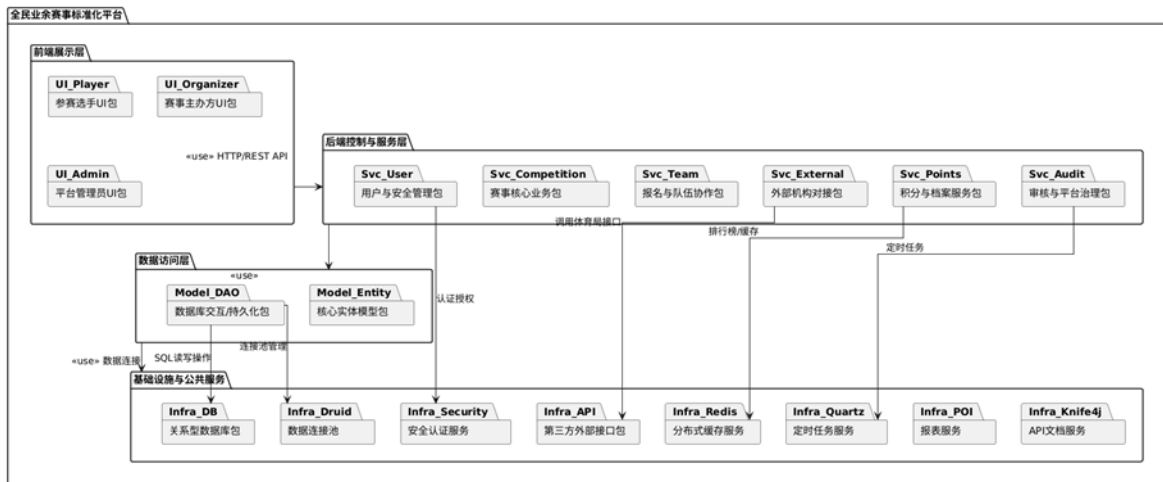
2. Apache POI

- 来源：<https://poi.apache.org>

- **获得方式:** Maven 依赖
 - **具体功能:** 提供报名名单、赛事成绩、统计报表的导入导出功能，支持批量处理 Excel 文档。
3. Quartz Scheduler
- **来源:** <https://quartz-scheduler.org>
 - **获得方式:** Maven 依赖
 - **具体功能:** 定时任务调度，用于赛事状态自动更新、积分定期结算、段位定时刷新及过期数据清理。
4. Knife4j
- **来源:** <https://gitee.com/xiaoym/knife4j>
 - **获得方式:** Maven 依赖
 - **具体功能:** 自动生成 API 接口文档，提供可视化在线调试界面，便于前端开发及体育局等外部机构对接调用。
5. Druid + Apache ShardingSphere
- **来源:** Druid (<https://github.com/alibaba/druid>)；ShardingSphere (<https://shardingsphere.apache.org>)
 - **获得方式:** Maven 依赖
 - **具体功能:** Druid 提供高性能数据库连接池、SQL 监控与防注入能力；ShardingSphere 提供数据分片与读写分离支持，为未来赛事数据的水平扩展预留能力。
6. Redis
- **来源:** <https://redis.io>
 - **获得方式:** 本地安装 Redis 服务器 + Maven 依赖（或 Python redis-py 库）
 - **具体功能:** 缓存热门赛事信息、用户会话、实时排行榜，降低数据库读取压力，提升系统响应速度。

四、确定基础设施和服务后的软件体系结构

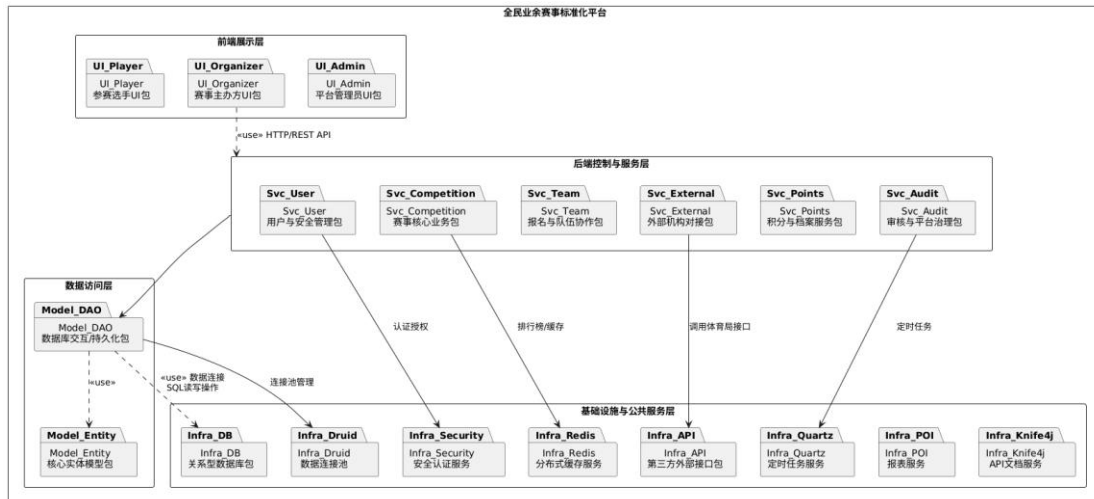
基础设施与公共服务层是系统运行的底层支撑，提供数据存储、安全认证、任务调度、数据访问、第三方对接等能力，保证系统高效、稳定、安全运行。



1. **关系型数据库包 (Infra_DB)**：存储用户、赛事、队伍、报名、积分、档案、审核记录等核心业务数据，保证数据持久化与一致性。
2. **第三方外部接口包 (Infra_API)**：提供与体育局、权威体育组织等外部机构的接口对接能力，实现赛事官方认证、数据互通。
3. **安全认证服务 (Infra_Security)**：基于 Spring Security + JWT 实现用户登录、身份校验、角色权限控制，确保系统访问安全。
4. **分布式缓存服务 (Infra_Redis)**：缓存热门赛事、用户会话、实时排行榜，提升系统响应速度，减轻数据库压力。
5. **定时任务服务 (Infra_Quartz)**：用于赛事状态自动更新、积分定期结算、排名刷新、过期数据清理等自动化任务。
6. **数据连接池 (Infra_Druid)**：提供数据库连接管理、SQL 监控、防注入，保障数据访问高效安全。
7. **报表服务 (Infra_POI)**：提供报名名单、赛事成绩、统计报表的导入导出功能。
8. **API 文档服务 (Infra_Knife4j)**：自动生成接口文档，支持在线调试，便于外部机构对接。

五、精化软件体系结构后的设计元素

架构采用分层设计，自上而下分为前端展示层、后端控制与服务层、数据访问层以及基础设施与公共服务层。



5.1 前端展示层

该层负责与不同角色的用户进行交互，通过 HTTP/REST API 统一与后端服务进行通信。

- **参赛选手 UI 包 (UI_Player)**: 面向普通参赛选手的用户界面。
- **赛事主办方 UI 包 (UI_Organizer)**: 面向赛事组织者、机构的用户界面，用于赛事的发布与管理。
- **平台管理员 UI 包 (UI_Admin)**: 面向平台内部运营和管理人员的用户界面。

5.2 后端控制与服务层

该层包含系统的核心业务逻辑，向下调用数据访问层和基础设施服务来实现各项业务功能。

- **用户与安全管理包 (Svc_User)**: 负责用户管理，并与底层的 Infra_Security 交互以实现认证授权。
- **赛事核心业务包 (Svc_Competition)**: 处理赛事创建、流程控制等核心业务，依赖 Infra_Redis 实现排行榜和数据的缓存调度。
- **报名与队伍协作包 (Svc_Team)**: 处理参赛队伍的组建、人员协作及赛事报名逻辑。
- **外部机构对接包 (Svc_External)**: 负责业务层的外部联络，通过调用底层 Infra_API 实现调用体育局等机构的接口。
- **积分与档案服务包 (Svc_Points)**: 管理选手的赛事积分、等级评定及个人档案。
- **审核与平台治理包 (Svc_Audit)**: 负责平台内的内容和资质审核，并可依赖 Infra_Quartz 触发定时的自动化审核或治理任务。

5.3 数据访问层

该层专职负责与数据库的交互和对象关系映射 (ORM)。

- **核心实体模型包 (Model_Entity)**: 定义系统中各个业务对象的实体类和数据结构。
- **数据库交互/持久化包 (Model_DAO)**: 处理数据的持久化操作，通过底层数据连接池进行连接池管理，并通过数据连接对 Infra_DB 进行 SQL 的读写操作。

5.4 基础设施与公共服务层

该层详细内容参见第 4 部分“确定基础设施和服务后的软件体系结构”。

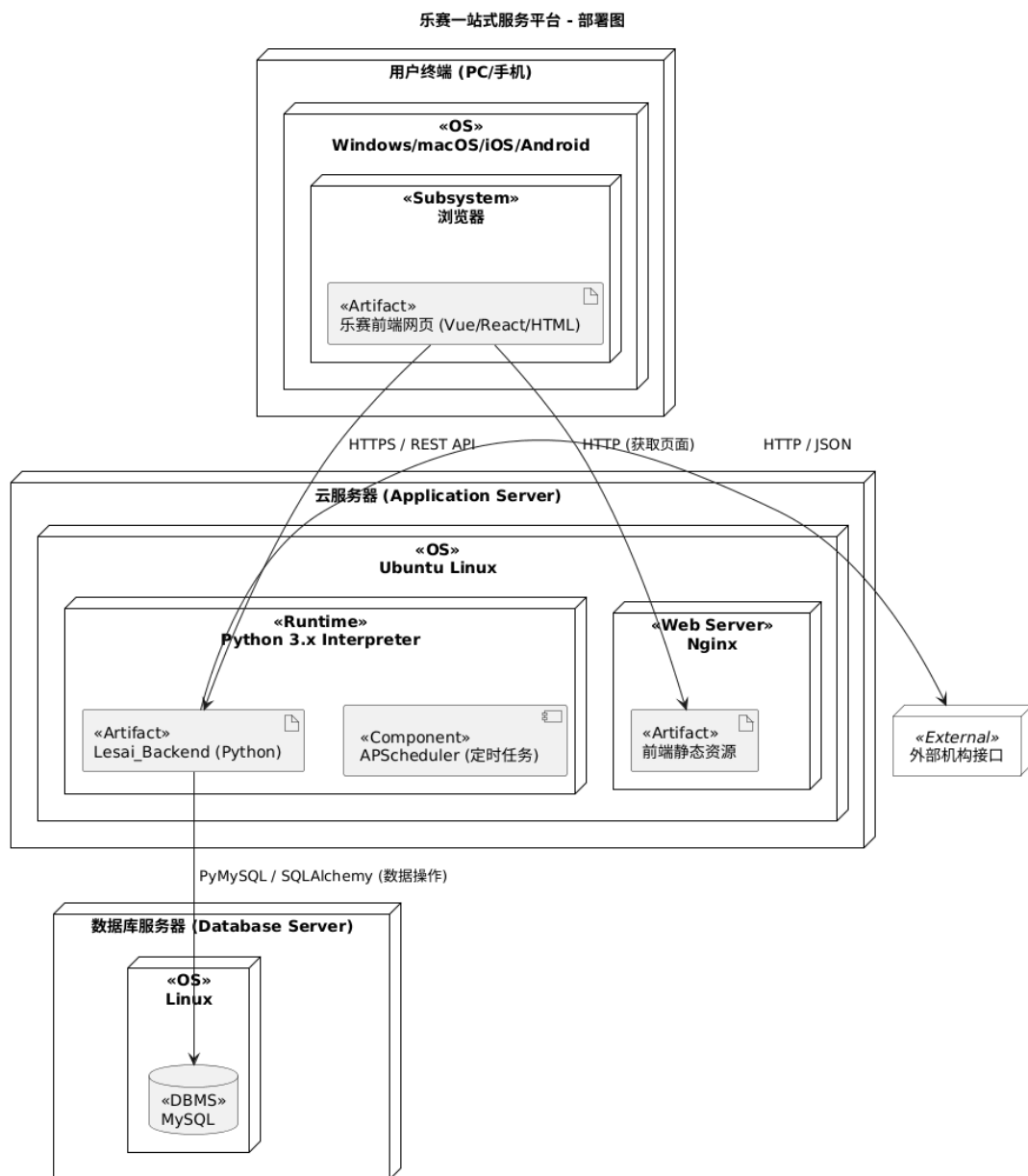
六、部署模型

6.1 设计说明：异构技术栈架构

架构设计说明：本系统在技术选型上采用“异构微服务”设计思想。后端核心业务逻辑（如报名管理、积分计算、赛事流程控制）采用 **Python** 语言开发，利用其灵活性与丰富的生态快速响应业务需求变化；基础设施层（安全认证、连接池监控、定时调度、API 文档生成）则复用 **Java** 生态中成熟稳定的开源组件（Spring Security、Quartz、Druid、Knife4j 等），两者通过标准 HTTP/RESTful API 进行跨语言通讯协作。该设计兼顾了业务开发的敏捷性与基础设施的健壮性，是工程实践中常见的技术组合策略。

6.2 部署架构描述

“乐赛一站式服务平台”采用轻量化的分布式 Web 部署架构。



用户终端部分：

该系统为纯网页端应用。参赛选手、主办方及管理员通过各类终端（PC 或移动设备）的浏览器访问平台。前端构件运行在浏览器环境中，通过 HTTPS 协议向后端发送 RESTful 风格的业务请求。

应用服务器部分：

部署在 Ubuntu Linux 云服务器上，通过动静分离的方式提高响应速度：

- **Nginx 服务器：**负责托管前端生成的静态资源（HTML/JS/CSS），确保用户能快速打开网页界面。
- **Python 运行环境：**核心后端程序（Lesai_Backend）运行在 Python 3.x 解释器中。它负责处理所有业务逻辑（如报名管理、积分计算等）。

- **Java 基础设施服务：**安全认证、定时调度、连接池监控等 Java 组件以独立服务或内嵌 Jar 形式运行，与 Python 业务后端通过内部 HTTP API 协同工作。
- **调度构件：**集成 APScheduler 或 Quartz 构件，用于实现赛事自动开启、积分定期结算等自动化定时任务。

数据库服务器部分：

系统使用 MySQL 作为核心数据库，独立部署在数据节点。它负责持久化存储用户信息、赛事数据、报名记录及积分档案，通过 Python 的 PyMySQL/SQLAlchemy 驱动及 Java 的 JDBC 提供稳定透明的数据存取支持。

外部协作部分：

系统通过互联网连接体育局等外部机构的 API 接口，实现官方赛事认证与数据的合规性交互。